

Aula 03 - Consumo Assíncrono e Operação de Consulta (GET)

Descrição: Implementação de requisições assíncronas em JavaScript (fetch, Promises, async/await) para consultar e renderizar dinamicamente os dados de encomendas no painel Front-End.

Abertura - Revisão

Antes de seguirmos em frente, vamos lembrar rapidinho o que já construímos nas duas últimas aulas. Essas perguntas podem ser respondidas em voz alta, com a turma toda participando:

- O que é uma API? (Dica: pense no garçom do restaurante da Aula 01.)
- Qual é a diferença entre Cliente e Servidor na arquitetura Client-Server?
- O que uma requisição HTTP carrega? E uma resposta?
- O que significam as famílias de status code 2xx, 4xx e 5xx?

Hoje damos o próximo passo: vamos fazer o JavaScript conversar de verdade com um servidor e atualizar a tela sozinho — sem apertar F5. É exatamente esse comportamento que falta no painel de entregas da nossa Situação de Aprendizagem, que hoje “trava sem retorno ao usuário” quando busca os pedidos. Vamos consertar isso.

Tópico 1 — O Problema do Código Bloqueante

1.1. A fila única da cantina

Imagine a cantina da escola com um único atendente. Se, para cada pedido, esse atendente precisasse ir até a cozinha, esperar o lanche ficar pronto parado ali, e só depois voltar para atender o próximo aluno da fila, o intervalo inteiro ia travar numa fila enorme.

É basicamente assim que o JavaScript funciona: ele é single-threaded, ou seja, só faz uma coisa de cada vez, numa pilha de execução (a Call Stack). Se uma dessas tarefas for lenta — como esperar a resposta de um servidor pela internet — e o código for escrito de forma bloqueante (síncrona), a página inteira congela até a resposta chegar.

1.2. Por que isso quebra a experiência do usuário

Voltando ao contexto da nossa Situação de Aprendizagem: o painel de entregas atual trava a tela sem dar retorno ao usuário justamente porque não trata esse tempo de espera de forma adequada. Ninguém quer clicar em “Ver meus pedidos” e ver a tela inteira congelar até a resposta do servidor chegar.

1.3. A solução: delegar e ser avisado depois

A solução do navegador é parecida com dar o pedido para a cozinha e continuar atendendo outros alunos: o JavaScript delega a tarefa lenta (a requisição de rede) para as Web APIs do navegador e continua executando o resto do código normalmente. Quando a resposta chegar, um mecanismo chamado Event Loop entrega o resultado de volta para o programa, sem travar nada.

Resumindo em uma frase: Programação assíncrona não significa que várias coisas acontecem ao mesmo tempo dentro do JavaScript — significa que tarefas lentas são delegadas e o resultado é entregue depois, sem travar a tela.

Tópico 2 — Promises: o Comprovante da Operação

2.1. A senha do totem de autoatendimento

Pense num totem de autoatendimento de lanchonete: você faz o pedido e, na hora, recebe uma senha (um comprovante) — não o lanche pronto. Essa senha representa uma promessa: em algum momento ela vai virar “lanche pronto” (sucesso) ou, se faltar algum ingrediente, um aviso de “pedido cancelado” (erro).

No JavaScript, esse “comprovante” se chama **Promise** (promessa). Toda Promise pode estar em um de três estados:

- **pending (pendente)** — a operação ainda não terminou. É a senha na mão, esperando ser chamada.
- **fulfilled (realizada)** — deu tudo certo, o resultado está disponível.
- **rejected (rejeitada)** — algo deu errado durante a operação.

2.2. Uma linha do tempo das soluções

Antes das Promises, o JavaScript resolvia tarefas assíncronas com **callbacks** (funções passadas como parâmetro para serem chamadas quando o dado chegasse). Funcionava, mas em fluxos mais longos o código virava uma verdadeira “escadinha” difícil de ler. As Promises organizaram isso, e o `async/await` — que veremos a seguir — deixou tudo ainda mais parecido com um código comum, de cima para baixo.

Abordagem	Como funciona (resumo)	Ponto forte	Ponto fraco
Callback	Passa uma função para ser executada quando o dado chegar	Simples e suportado em qualquer navegador	Muitas funções aninhadas ficam difíceis de ler
Promise (.then/.catch)	Devolve um objeto que avisa quando a tarefa termina, com sucesso ou erro	Encadeamento mais organizado que o callback	Ainda fica extenso em fluxos com várias etapas
async / await	Permite escrever código assíncrono parecido com código comum, de cima para baixo	Leitura simples e uso natural do try/catch	Só funciona dentro de uma função declarada como async

Repare que cada abordagem não substitui totalmente a anterior — o `async/await`, que vamos usar o resto da aula, é construído em cima das Promises. Toda função `fetch()` devolve uma Promise por baixo dos panos.

Tópico 3 — async/await na Prática

A sintaxe `async/await` é uma forma de “ler” uma Promise como se fosse um código comum, sem precisar encadear vários `.then()`. Duas palavras-chave fazem toda a mágica:

- **async** — colocada antes de function, transforma a função para que ela sempre devolva uma Promise automaticamente.

- **await** — só pode ser usada dentro de uma função async. Ela pausa a execução daquela função (só dela, o resto da página continua funcionando normalmente) até a Promise ser resolvida.

Exemplo — testando no Console do navegador (F12):

Para testar a sintaxe `async/await` de forma simples e direta sem precisar criar nenhum arquivo HTML ou servidor local, você pode executar o código diretamente no Console do Navegador (DevTools).

1. Abra o Console em qualquer página

- Acesse a página <https://brasilapi.com.br/> no navegador.
- Pressione a tecla F12 no seu teclado (ou clique com o botão direito na página > Inspecionar).
- Selecione a aba Console na janela que se abrir.
- Para liberar a colagem de código, digite “permitir colar” no console.

2. Cole e execute o código

- Copie o bloco abaixo, cole dentro do Console e pressione Enter:

```
async function buscarDados() {  
  const resposta = await fetch("https://brasilapi.com.br/api/ddd/v1/47");  
  const dados = await resposta.json();  
  console.log("Dados da API:", dados);  
}
```

```
buscarDados();  
console.log("Essa linha aparece ANTES dos dados, pois a tela não travou!");
```

3. Observe o comportamento assíncrono

- A frase `"Essa linha aparece ANTES dos dados..."` será impressa imediatamente no console.
- Poucos milissegundos depois, quando a internet responder, o objeto contendo as informações da API (cidades do DDD 47) aparecerá na tela.

O que este teste comprova na prática:

- Execução Não-Bloqueante: A execução do programa continuou normalmente enquanto a requisição de rede era processada.
- Uso do `await`: O `await` só congelou o fluxo interno da função `buscarDados()` até que a requisição terminasse, sem travar a interface nem impedir outras linhas de rodarem.

Erro comum: `await` só existe dentro de uma função async. Se você tentar usar `await` “solto” fora de uma função async, o navegador vai acusar erro de sintaxe. Sempre confira o cabeçalho da função antes de usar `await` dentro dela.

Repare, no exemplo acima, que a mensagem do `console.log` final aparece antes dos dados da API — isso prova, na prática, que o navegador não ficou parado esperando a resposta chegar.

Tópico 4 — A Fetch API e a Operação GET

4.1. Fazendo a primeira chamada

A Fetch API é a ferramenta padrão do navegador para fazer requisições HTTP. Por padrão, ao chamar `fetch(url)` sem configurações extras, o navegador realiza uma requisição GET — exatamente a operação de Consulta do ciclo CRUD que estamos trabalhando hoje.

`fetch()` devolve uma Promise que resolve para um objeto Response — uma espécie de envelope da resposta HTTP. Para ler o conteúdo (o corpo, ou body) desse envelope no formato JSON, chamamos `response.json()`, que também é assíncrono e devolve outra Promise.

4.2. A armadilha do `response.ok`

Atenção — isso derruba muitos projetos: A Promise do `fetch()` SÓ é rejeitada quando há uma falha real de rede (sem internet, servidor fora do ar, bloqueio de CORS). Se o servidor responder normalmente, mesmo que seja com um erro HTTP como 404 ou 500, o `fetch` considera que “deu certo”, porque a mensagem chegou até o navegador!

Por isso, é obrigatório verificar manualmente a propriedade `response.ok` antes de continuar. Lembra dos status codes da Aula 02? `response.ok` só é `true` quando o status está entre 200 e 299

Situação	Status HTTP	<code>response.ok</code>	O que fazer
Deu tudo certo	200	<code>true</code>	Ler os dados com <code>.json()</code> e mostrar na tela
Cliente enviou algo errado	400	<code>false</code>	Avisar o usuário que os dados informados são inválidos
Rota não existe	404	<code>false</code>	Avisar que o recurso não foi encontrado
Servidor apresentou falha	500	<code>false</code>	Avisar sobre falha temporária e sugerir tentar novamente

4.3. Filtrando a consulta com Query Parameters

Lembra da Aula 02, quando vimos que parâmetros de uma requisição GET vão anexados na URL depois do símbolo `??`? A interface `URLSearchParams` monta essa parte da URL de forma segura, cuidando automaticamente de espaços e caracteres especiais. Isso será muito útil quando adicionarmos filtros ao nosso painel (por cliente, por status, etc.).

Exemplo — montando uma URL com filtro:

```
const url = new URL("https://api.exemplo.com/encomendas");
const filtros = new URLSearchParams();
filtros.append("status", "PENDENTE");
url.search = filtros.toString();

console.log(url.toString());
// https://api.exemplo.com/encomendas?status=PENDENTE
```

Tópico 5 — Prática Guiada: Construindo o Painel de Encomendas

Agora vamos aplicar tudo o que vimos hoje para começar a construir, de verdade, o painel da nossa Situação de Aprendizagem. Este código será a base que vamos evoluir nas próximas aulas (Criação, Atualização, Exclusão, tratamento de falhas e autenticação).

Passo 1 — Preparar o ambiente

- Crie uma pasta chamada painel-encomendas.
- Dentro dela, crie três arquivos: index.html, app.js e encomendas.json.
- Abra a pasta no VS Code e instale (se ainda não tiver) a extensão Live Server.

Por que não posso simplesmente abrir o arquivo clicando duas vezes?

Lembra da Aula 02? O fetch precisa de um ambiente de rede (http:// ou https://) para funcionar — arquivos abertos direto pelo protocolo file:// não executam a arquitetura de comunicação do HTTP corretamente. Por isso vamos usar o Live Server, que cria um servidor local para o nosso projeto.

Passo 2 — Montar o HTML base

index.html

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Painel de Encomendas</title>
</head>
<body>
  <h1>Painel de Controle de Entregas</h1>

  <p id="carregando">Carregando encomendas...</p>
  <p id="erro" style="color: red; display: none;"></p>

  <table id="tabela-encomendas" border="1">
    <thead>
      <tr>
        <th>Código</th>
        <th>Cliente</th>
        <th>Status</th>
      </tr>
    </thead>
    <tbody id="corpo-tabela"></tbody>
  </table>
```

```
<script src="app.js"></script>
</body>
</html>
```

Passo 3 — Simular os dados do servidor

Como a API real da empresa de logística ainda não está disponível para testes, vamos simular o servidor com um arquivo JSON local. A estrutura dos dados já é a mesma que usaremos mais adiante com a API de verdade — quando ela estiver disponível, basta trocar a URL do fetch.

encomendas.json

```
[
  { "codigoRastreio": "BR001", "clienteNome": "Ana Souza", "status": "EM_TRANSITO" },
  { "codigoRastreio": "BR002", "clienteNome": "Carlos Lima", "status": "ENTREGUE" },
  { "codigoRastreio": "BR003", "clienteNome": "Fernanda Dias", "status": "PENDENTE" }
]
```

Passo 4 — Escrever a função de consulta (GET)

Esta função vai buscar os dados de forma assíncrona, verificar se a resposta foi bem-sucedida e chamar a renderização na tela:

app.js — parte 1

```
const elCarregando = document.getElementById("carregando");
const elErro = document.getElementById("erro");
const corpoTabela = document.getElementById("corpo-tabela");
```

```
async function consultarEncomendas() {
  elCarregando.style.display = "block";
  elErro.style.display = "none";

  try {
    const resposta = await fetch("encomendas.json");

    if (!resposta.ok) {
      throw new Error("O servidor respondeu com erro: " + resposta.status);
    }

    const encomendas = await resposta.json();
    renderizarTabela(encomendas);

  } catch (erro) {
    elErro.textContent = "Não foi possível carregar as encomendas. Tente novamente.";
    elErro.style.display = "block";
    console.error(erro);
  }
}
```

```
} finally {  
    elCarregando.style.display = "none";  
}  
}  
  
function renderizarTabela(encomendas) {  
    corpoTabela.innerHTML = ""; // limpa a tabela antes de preencher de novo  
  
    const fragmento = document.createDocumentFragment();  
  
    encomendas.forEach(encomenda => {  
        const linha = document.createElement("tr");  
  
        const tdCodigo = document.createElement("td");  
        tdCodigo.textContent = encomenda.codigoRastreio;  
  
        const tdCliente = document.createElement("td");  
        tdCliente.textContent = encomenda.clienteNome;  
  
        const tdStatus = document.createElement("td");  
        tdStatus.textContent = encomenda.status;  
  
        linha.appendChild(tdCodigo);  
        linha.appendChild(tdCliente);  
        linha.appendChild(tdStatus);  
  
        fragmento.appendChild(linha);  
    });  
  
    corpoTabela.appendChild(fragmento);  
}  
  
consultarEncomendas(); // chama a função assim que a página carrega
```

Passo 5 — Renderizar os dados dinamicamente

Aqui está o coração da atualização dinâmica: em vez de recarregar a página, criamos os elementos da tabela diretamente no DOM com JavaScript. Usamos `createElement` e `textContent` — e não `innerHTML` — como boa prática de segurança sempre que exibimos dados vindos do servidor (aprofundaremos esse cuidado na Aula 07, sobre segurança da informação).

Passo 6 — Testar no navegador

- Clique com o botão direito em `index.html` e selecione “Open with Live Server”.
- Abra o DevTools (F12) e vá até a aba Network (Rede).

- Recarregue a página e observe a requisição para encomendas.json: confira o método (GET) e o status (200).
- Confirme que a tabela foi preenchida com os três registros do arquivo JSON.
- **Desafio extra:** renomeie temporariamente o arquivo encomendas.json e recarregue a página — observe o tratamento de erro funcionando.

Atividade - Checkpoint de Consulta

Escolha **UMA** das APIs públicas abaixo (já exploradas nas outras aulas) e construa uma pequena página que consulte e exiba os dados dinamicamente na tela, seguindo o mesmo padrão da prática guiada:

- BrasilAPI — CEP: <https://brasilapi.com.br/api/cep/v1/{cep}>
- BrasilAPI — DDD: <https://brasilapi.com.br/api/ddd/v1/{ddd}>
- BrasilAPI — Feriados Nacionais: <https://brasilapi.com.br/api/feriados/v1/{ano}>

Requisitos mínimos

- Usar `fetch()` dentro de uma função `async`, com `await`.
- Verificar `response.ok` antes de tratar o resultado como sucesso.
- Renderizar o resultado no DOM usando `createElement/textContent`.
- Mostrar algum feedback simples de carregamento (ex.: um texto “Carregando...”).

Critérios de Avaliação

Critério	Atende Plenamente	Atende Parcialmente	Não Atende
Uso de <code>fetch</code> + <code>async/await</code>	Implementado corretamente, sem erros de sintaxe	Implementado, mas com pequenos erros	Não implementado ou usa apenas <code>.then()</code> sem justificativa
Verificação de <code>response.ok</code>	Verifica e trata o erro adequadamente	Verifica, mas não trata o erro	Não verifica
Renderização dinâmica no DOM	Usa <code>createElement/textContent</code> corretamente	Usa, mas com <code>innerHTML</code> em dados vindos do servidor	Não renderiza dinamicamente
Feedback ao usuário	Mostra estado de carregamento e de erro	Mostra apenas um dos dois	Não mostra nenhum feedback

Entrega: arquivo HTML + JS (pode ser um único arquivo) enviado no AVA até o final da aula. Este checkpoint compõe a primeira parte do que será completado com as operações de Criação, Atualização e Exclusão nas Aulas 04 e 05.

Fechamento

Hoje aprendemos a fazer o JavaScript conversar com um servidor sem travar a tela: entendemos por que o código bloqueante é um problema, como as Promises representam operações que ainda vão terminar, como

o `async/await` deixa esse código mais legível, e como usar a Fetch API para realizar a primeira operação do ciclo CRUD — a Consulta (GET).

Próxima aula (Aula 04): Envio de Dados e Operação de Criação (POST). Vamos construir um formulário integrado à API, serializar os dados em JSON e validar o retorno de criação (status 201) — a segunda peça do ciclo CRUD do nosso painel.

Guardem bem os arquivos do painel-encomendas construído hoje: eles serão a base para todas as próximas aulas até o Projeto Integrador.

Glossário Rápido

Termo	O que significa
Promise	Um “comprovante” que representa uma operação assíncrona que ainda vai terminar, com sucesso ou erro
async	Palavra-chave que transforma uma função para que ela use <code>await</code> e devolva uma Promise automaticamente
await	Pausa a execução da função (só ela) até que a Promise seja resolvida
fetch()	Função do navegador usada para realizar requisições HTTP
endpoint	O endereço específico de um recurso disponibilizado pela API
JSON	Formato de texto usado para organizar dados em pares chave/valor
DOM	Estrutura em árvore que representa a página HTML na memória do navegador
response.ok	Propriedade que indica se o status HTTP da resposta está entre 200 e 299
Query Parameters	Filtros enviados na URL, depois do símbolo ?